

TRABALHO FINAL DE ARQUITETURA: ANÁLISE DE MEMÓRIA CACHE

Breno Demio Padilha

Graduando(a) em Ciência da Computação

Paola Siqueira Cecconello

Graduando(a) em Ciência da Computação

Rafaela Padilha Leitoles

Graduando(a) em Ciência da Computação

1 INTRODUÇÃO

A memória cache é a memória comumente utilizada pelo processador para acessar e manipular dados das memórias principal e virtual mais rapidamente. Por ser mais veloz, para que seu valor de mercado não seja demasiado elevado, seu tamanho é reduzido e pode ser distribuído em diferentes níveis, chamados L1, L2, L3, etc. Consequentemente, por ser menor, ela possui menos endereços, o que pode resultar em um *cache miss*, ou seja, a falta de um dado requisitado pelo processador. Quando este evento ocorre, é necessário que haja uma troca de blocos entre as memórias, tornando o processo consideravelmente mais lento. No entanto, a memória cache conta com um princípio denominado “princípio da localidade”, o qual diminui as faltas, mas não as elimina por completo. A partir disso, será desenvolvido um estudo acerca do desempenho da memória cache, iniciando com a determinação de seu tamanho e seguindo com testes previamente estabelecidos.

2 DESENVOLVIMENTO

2.1 Descobrindo o tamanho da memória cache

Há duas formas de descobrir o tamanho da linha/bloco da memória cache e dos diferentes níveis de tal: via comandos no terminal e via testes empíricos. Usando os comandos no terminal, o sistema operacional apenas lê os dados do processador, enquanto usando testes empíricos, a informação é descoberta por meio da análise de dados coletados usando um teste de benchmark. Ambas serão apresentadas mais profundamente no decorrer desse artigo.

2.1.1 Via comando:

Em sistemas operacionais Linux, informações sobre a CPU e a cache podem ser facilmente encontradas no diretório `sys`. Dessa forma, nos diretórios `indexN`, localizados em `cpu/cpu0/cache`, é possível realizar uma análise de diversos aspectos internos, sendo N o número de identificação da respectiva cache. Nos testes feitos e apresentados a seguir, serão utilizadas as máquinas do Departamento de Informática, no entanto, é necessário considerar que os resultados variam para cada *hardware*. No exemplo que segue, o “index0” é a cache L1 que armazena os dados que o programa utiliza, “index1” é a cache L1 que guarda as instruções do programa, “index2” é a L2 e o “index3” é a L3, sendo que estes dois últimos armazenam tanto dados quanto instruções (unified). Analisando o conteúdo de “coherency_line_size”, é possível determinar o tamanho de uma linha da memória cache.

Shell

```
cat /sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size
```

Comando digitado no terminal para obter o resultado

O tamanho de todas as linhas, seja da L1, L2 e L3, resultou em 64 bytes.

Além disso, como dito anteriormente, é possível analisar outros aspectos:

Shell

```
bdp25@macalan:/sys/devices/system/cpu/cpu0/cache/index0 $ ls
coherency_line_size id level number_of_sets physical_line_partition
shared_cpu_list shared_cpu_map size type uevent ways_of_associativity
```

Listagem de todos os arquivos de `/sys/devices/system/cpu/cpu0/cache/index0`

- **level:** Indica qual é o nível da cache (L1,L2,L3);
- **type:** Indica se guarda dados, instruções, ou os dois (unified);
- **number_of_sets:** Indica quantas linhas/blocos a cache possui;
- **ways_of_associativity:** Indica quantas vias tem a cache;
- **size:** Indica o tamanho total da cache.

Com essas informações, é possível verificar o tamanho da cache manualmente, como no exemplo a seguir:

Shell

```
bdp25@macalan:/sys/devices/system/cpu/cpu0/cache/index0 cat coherency_line_size
64
bdp25@macalan:/sys/devices/system/cpu/cpu0/cache/index0 cat number_of_sets
64
bdp25@macalan:/sys/devices/system/cpu/cpu0/cache/index0 cat
ways_of_associativity
8
bdp25@macalan:/sys/devices/system/cpu/cpu0/cache/index0 cat size
32K
```

Realizando o cálculo: $64 \times 64 = 4096 \times 8 = 32768 = 32K$, para a L1.

Fazendo esse mesmo cálculo para index2 e 3 (L2 e L3), obtém-se os valores 4096K e 16384K, respectivamente, o que encaixa com a hierarquia de memória, ou seja, caches mais distantes do processador possuem maior tamanho.

Desse modo, o conteúdo dos diretórios informa que esses são os valores da cache, mas não há garantia efetiva nestes dados, considerando que não foram testados. Portanto, é interessante provar que esses valores estão corretos, testando e descobrindo a capacidade da memória cache a partir de testes empíricos.

2.1.2 Via testes empíricos:

Os testes empíricos para descobrir tanto o tamanho da linha da memória cache quanto o tamanho total da mesma consistem em analisar o tempo necessário para percorrer vetores de tamanhos variados, considerando que o tamanho do vetor em que ocorre uma mudança de tempo repentina indica o tamanho da linha ou cache em estudo. Apesar da lógica que fundamenta os testes empíricos serem semelhantes, cada um detém especificidades que serão apresentadas.

Para realizar o teste empírico do tamanho da linha da cache, primeiramente é criado um vetor de um tamanho fixo que certamente é maior que uma memória cache, por exemplo, 256 MB, e em seguida é definido o valor inicial do *stride* como de 1 byte, sendo *stride* o termo usado para definir o intervalo de acessos sequenciais na memória. Assim, o vetor é percorrido com *stride* de tamanho 1 byte e o tempo gasto neste processo é registrado, possibilitando a coleta de dados. O processo é realizado sequencialmente dobrando o tamanho do *stride* até que esse tenha tamanho 256 MB. Ao final da coleta do tempo gasto para percorrer o vetor com diferentes tamanhos de *strides* o comportamento esperado divide os dados em

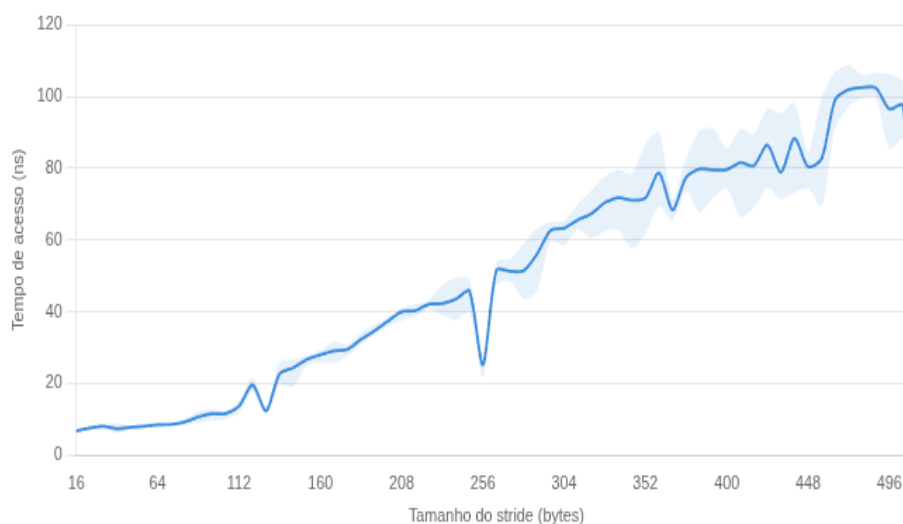
duas partes, onde nos primeiros testes o tempo é curto e estável e nos últimos o tempo é mais longo e também estável. O tamanho do *stride* durante a transição entre esses dois comportamentos é o indicador do tamanho da linha da cache, ou seja, na primeira etapa, o tempo curto e estável representa a memória acessando diversos endereços na mesma linha da cache. Por essa já estar carregada na memória, o tempo para acesso é curto. Já quando o tamanho do *stride* é o mesmo ou maior que a da linha da cache, isso força o carregamento de uma nova linha, aumentando o tempo. Esse comportamento pode ser observado no teste executado utilizando uma benchmark, feita pelo professor da Universidade de Quebec Daniel Lemire, que avaliou o tamanho da linha cache da Macallan, máquina virtual do servidor do DINF. Os resultados são os seguintes, considerando apenas *strides* com tamanho de potências de 2:

Shell

```
bdp@bdp-desktop:~$ ./cacheline
```

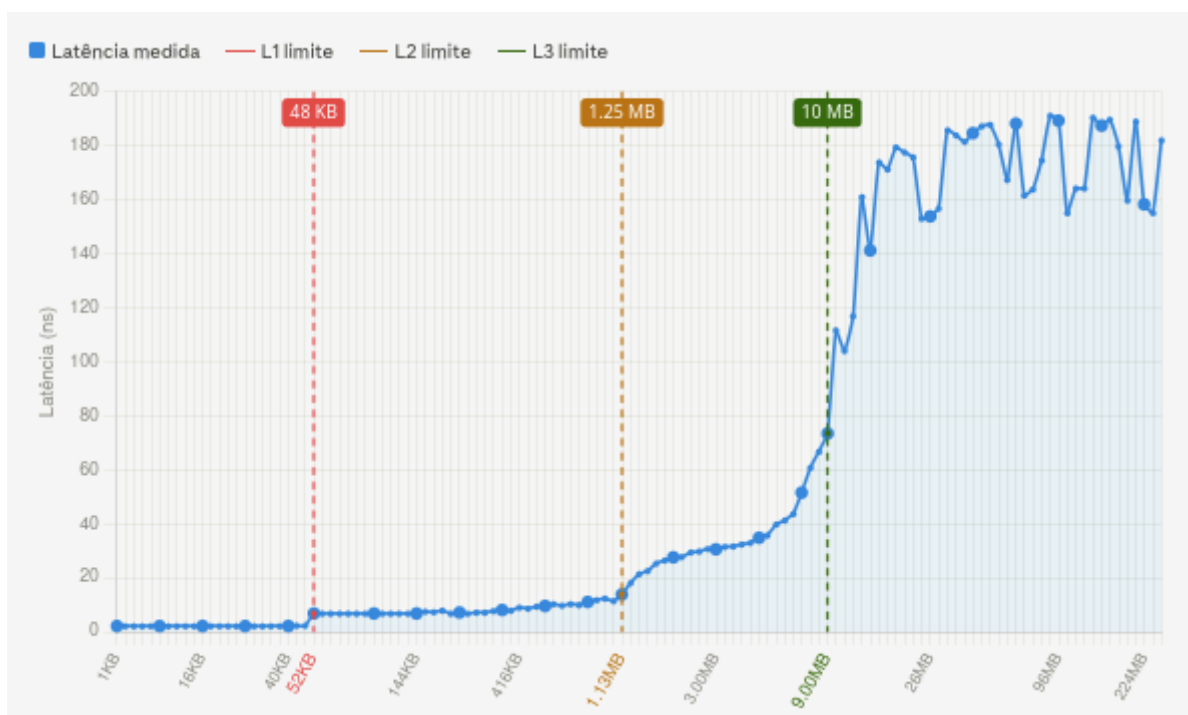
stride size	average	min	max
16	7.0	6.3	7.2
32	8.5	8.4	8.6
64	7.8	7.6	7.8
128	11.7	11.5	11.8
256	22.7	22.2	22.9
512	36.4	36.0	36.8

Nesse exemplo, é possível perceber que o aumento começa a surgir apenas após o stride de tamanho 64, portanto, só pode significar que 64 é a quantidade de linhas da cache, visto que antes o tempo era baixo e estável, e após o 64 começa o aumento no tempo, como o gráfico aponta:



O teste empírico para descobrir o tamanho das memórias caches é similar ao teste para descobrir o tamanho da linha da cache. Nesse caso, o *stride* é fixo em 64 bytes, já que esse é o tamanho da linha da cache, e o tamanho do vetor começa com um valor considerado pequeno, por exemplo, 1KB. Assim, o tempo necessário para percorrer esse vetor de 1KB em intervalos de 64 bytes é medido e o processo é repetido dobrando o tamanho do vetor e percorrendo-o até que este seja um provável tamanho de cache, por exemplo, 256MB. O tempo gasto para cada um desses processos é coletado e, ao final, analisando os tempos obtidos, é apresentado um comportamento que pode ser dividido em três partes semelhantes entre si, sendo cada um referente a um nível de cache diferente. Basicamente, os primeiros tempos coletados são constantes e mais baixos que os demais, esses permanecem assim até um determinado tamanho de vetor, que ao ser percorrido, ocorre um aumento no tempo de acesso, indicando o acesso a um novo nível de memória cache. Essa mudança de nível só ocorre quando o limite do nível de cache anterior é atingido, portanto o tamanho desse vetor indica o tamanho da L1. A partir desse ponto, o tempo volta a ser constante, porém com valores mais altos que o último período de estabilidade. Esse padrão (um período de estabilidade no tempo, seguido de uma mudança repentina, e por fim a volta de uma estabilidade com um valor maior que o anterior) se repete para cada mudança de nível de cache, que ocorre quando o nível anterior não tem capacidade suficiente para todo o vetor, criando uma espécie de “escada”, onde o tamanho do vetor associado a mudança do comportamento do tempo indica o tamanho do nível da cache.

Considerando o tamanho da cache, torna-se viável a realização dos testes de desempenho desta, os quais são baseados nos resultados da execução da benchmark “lmbench”, mais especificamente da execução do comando “lat_mem_rd”, que apresenta como resultado o tamanho do *buffer* lido, ou seja, o tamanho do vetor, e o tempo de latência referente ao tempo de acesso deste, assim, ao organizar essas informações em um gráfico, em que o eixo das abscissas representa o tamanho do *buffer* e no eixo das ordenadas a latência, obtém-se:



É possível notar o fenômeno dos “degraus” no gráfico nos pontos onde os valores do *buffer* valem aproximadamente 48KB, 1.25MB, 10MB, que correspondem aos tamanhos de L1, L2 e L3 respectivamente. Nota-se a transição ruidosa entre a latência da L3 e da RAM, isso acontece pois esse processo é marcado pela interferência do sistema operacional, que busca otimizar o tempo de acesso a RAM. Nota-se também a disparidade com o resultado obtido via comando, isso se deve pois os testes ocorreram em diferentes computadores com diferentes configurações.

2.2 Analisando a performance da memória cache

Para fazer a análise da performance da memória cache serão utilizados dois métodos de multiplicação de matrizes, sendo o primeiro o método casual e o segundo separando as matrizes a serem multiplicadas em 4 blocos. A escolha do experimento utilizou como base o fato de a divisão de uma matriz em submatrizes tornar os acessos à memória menos frequentes, visto que as operações a serem realizadas sobre uma submatriz baseiam-se em processos repetitivos, com endereços próximos, mantendo-os na memória cache durante a execução e diminuindo a taxa de *cache miss*.

2.2.1 Teste de performance implementado:

Assim, para a realização do teste foram necessários dois programas, desenvolvidos na linguagem C, que executam a mesma tarefa: a multiplicação de uma matriz por ela mesma. A implementação utilizou uma matriz quadrada de tamanho 512x512, para que as diferenças nos resultados fossem mais relevantes.

A primeira implementação baseou-se no método tradicional de multiplicação de matrizes, considerando o seguinte diagrama:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}_{3 \times 2} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}_{2 \times 2} = \begin{bmatrix} a_{11} \cdot b_{11} + a_{12} \cdot b_{21} & a_{11} \cdot b_{12} + a_{12} \cdot b_{22} \\ a_{21} \cdot b_{11} + a_{22} \cdot b_{21} & a_{21} \cdot b_{12} + a_{22} \cdot b_{22} \\ a_{31} \cdot b_{11} + a_{32} \cdot b_{21} & a_{31} \cdot b_{12} + a_{32} \cdot b_{22} \end{bmatrix}_{3 \times 2}$$

Com isso, a implementação final resultou no seguinte algoritmo:

C/C++

```
#include <stdio.h>

#define N 512

int main()
{
    int matriz[N][N];
    int resultado[N][N] = {0};

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            matriz[i][j] = (i + j) % 100;

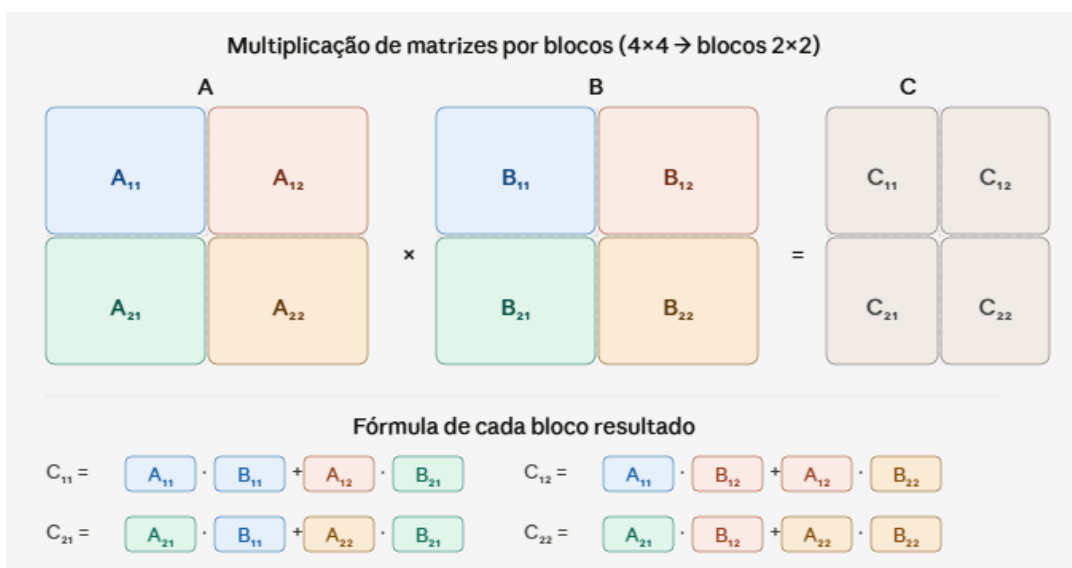
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            for (int k = 0; k < N; k++)
```

```

                                resultado[i][j] = resultado[i][j] +
matriz[i][k] * matriz[k][j];
                                printf("%d\n", resultado[N-1][N-1]);
                                return 1;
                                }

```

Já a segunda implementação, baseou-se em um método não muito conhecido que torna o processo mais eficiente. Este consiste em dividir cada matriz em blocos e, para obter o bloco C_{11} do resultado, por exemplo, é necessário multiplicar cada submatriz da linha 1 da primeira matriz por cada submatriz da coluna 1 da segunda matriz e, no final, somar todos os resultados obtidos. Para facilitar o entendimento, segue um diagrama visual:



Com base neste processo, foi desenvolvido o algoritmo a seguir:

```

C/C++
#include <stdio.h>

#define N 512

#define BLOCO 64

```



```

int main()
{
    int matriz[N][N];
    int resultado[N][N] = {0};

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            matriz[i][j] = (i + j) % 100;

    for (int i = 0; i < N; i += BLOCO)
        for (int j = 0; j < N; j += BLOCO)
            for (int k = 0; k < N; k += BLOCO)
                for (int ii = i; ii < i + BLOCO; ii++)
                    for (int jj = j; jj < j + BLOCO; jj++)
                        for (int kk = k; kk < k + BLOCO;
                            kk++)
                            resultado[ii][jj] +=
matriz[ii][kk] * matriz[kk][jj];
    printf("%d\n", resultado[N-1][N-1]);
}

```

É importante apontar a necessidade de tomar cuidado ao compilar esse código. Compiladores modernos observam padrões em códigos e conseguem otimizar o próprio código para o hardware rodar otimizado. Por isso a necessidade de colocar o “*printf*” com o resultado e compilar com:

```

Shell
gcc -O2 -fno-tree-vectorize -fno-unroll-loops -o m1 mul_matrizes.c
gcc -O2 -fno-tree-vectorize -fno-unroll-loops -o m2 mul_matrizes_blocos.c

```

Para entender os resultados finais, é necessário verificar como a adição dos blocos tornou o programa mais eficiente. Ao adicionar mais loops internos, o programa passou a utilizar endereços de memória mais próximos, ou seja, pertencentes a um mesmo bloco, além de repetir instruções mais vezes do que anteriormente, tornando a execução mais eficiente. Antes, o algoritmo percorria toda uma linha, passando por todas as colunas e depois voltava ao começo, o que causava mais *cache miss* por acesso de dois endereços distantes na memória. Com isso, é notória a diferença conceitual e prática dos processos apresentados.

2.2.2 Resultados observados:

M1 foi o método tradicional de multiplicação de matriz, com a matriz inteira, já M2 é o método dividindo em blocos:

```
Shell
perf stat --repeat 10 -e
task-clock,cycles,instructions,cache-references,cache-misses ./m1

Performance counter stats for './m1' (10 runs):

          315,84 msec task-clock                #    0,996 CPUs
utilized              ( +- 1,43% )
      1.157.602.766      cycles                  #    3,665 GHz
( +- 0,67% )
      952.609.771      instructions              #    0,82 insn per
cycle              ( +- 0,00% )
      251.263.543      cache-references          #   795,550 M/sec
( +- 0,06% )
      131.823.981      cache-misses              #   52,46% of all cache
refs              ( +- 0,70% )

      0,31702 +- 0,00454 seconds time elapsed ( +- 1,43% )
```

Resultados do método 1 obtidos com `perf stat --repeat 10 -e task-clock,cycles,instructions,cache-references,cache-misses ./m1`

m2:

```
Shell
perf stat --repeat 10 -e
task-clock,cycles,instructions,cache-references,cache-misses ./m2
```

Performance counter stats for './m2' (10 runs):

```

143,85 msec task-clock # 0,991 CPUs
utilized ( +- 0,62% )
519.165.658 cycles # 3,609 GHz
( +- 0,50% )
969.976.281 instructions # 1,87 insn per
cycle ( +- 0,00% )
202.063.959 cache-references # 1,405 G/sec
( +- 7,58% )
236.967 cache-misses # 0,12% of all cache
refs ( +- 3,15% )

0,145115 +- 0,000883 seconds time elapsed ( +- 0,61% )

```

Resultados do método 2 obtidos com `perf stat --repeat 10 -e task-clock,cycles,instructions,cache-references,cache-misses ./m2`

Para uma análise íntegra, utiliza-se o comando "`perf stat`" com a flag "`--repeat 10`" para conseguir uma média dos dados de performance da cache, sendo os valores em parênteses o desvio padrão dos resultados.

Métrica	M1 (tradicional)	M2 (blocos)	Variação (M2 vs M1)
Tempo de execução	315,84 ms	143,85 ms	-54,5%
Total de ciclos de clock	1.157.602.766	519.165.658	-55,1%
Instruções executadas	952.609.771	969.976.281	+1,8%
IPC (instruções/ciclo)	0,82	1,87	+128%
Cache-references	251.263.543	202.063.959	-19,6%
Cache-misses	131.823.981	236.967	-99,82%
Taxa de cache-miss	52,46%	0,12%	99,77%

Assim, é possível apontar alguns resultados interessantes. A diferença na quantidade de cache-misses é com certeza o dado principal. Ao utilizar o método em blocos ela é significativamente menor e mais eficiente: de 53% do método tradicional para incrivelmente apenas 0,12%, quase que não possuindo cache-misses, uma diferença de 556 vezes, que é uma evidência do princípio de localidade entrando em ação, dado que os loops for do código de M2 vão em blocos de 64, exatamente o tamanho da L1. Muito por conta disso, a cache na M2 não tem que esperar tanto a transferência de dados da memória principal, fazendo com que o programa rode 2.2 vezes mais rápido que o método tradicional. Outro fator significativo para se apontar é a similaridade nas cache-references, mostrando que o ganho de tempo da M2 não é de ter que acessar menos a cache, mas sim o tempo salvo que ele não gasta na demora que acontece quando você não encontra o que está procurando na cache. Por fim, é interessante mostrar o aumento do IPC, que não possui tantos stalls na M2 dado que não acontecem tantos cache misses, alavancando na M2.

3 CONSIDERAÇÕES FINAIS

O presente estudo demonstrou, de maneira prática, o funcionamento do “princípio da localidade” ao utilizar dois programas diferentes para realizar uma mesma tarefa: um acessando endereços de memória mais distantes, ocasionando mais *cache miss* e outro acessando endereços mais próximos. A partir dos resultados é possível analisar a diferença de performance da cache, considerando que o segundo programa apresentou uma taxa de erro na cache aproximadamente 500 vezes menor que o primeiro. É importante ressaltar que tal diferença se dá também pelo tamanho das matrizes utilizadas no experimento. Para matrizes menores, a diferença de desempenho é menor, enquanto para matrizes maiores, a diferença aumenta proporcionalmente, visto que os acessos à memória tornam-se mais frequentes.

Além disso, o experimento empírico realizado para identificar o tamanho da cache e do bloco da mesma demonstrou a confiabilidade dos documentos que armazenam as informações sobre o *hardware*. Com isso, é necessário destacar a importância da memória cache, principalmente nos computadores modernos, que buscam otimização e alto desempenho, sendo ela um elemento com possibilidade de exploração para novas tecnologias.

REFERÊNCIAS

LEMIRE, Daniel. Measuring the size of the cache line empirically. **Daniel Lemire's blog**. Disponível em:

<https://lemire.me/blog/2023/12/12/measuring-the-size-of-the-cache-line-empirically/>.

Acesso em: 05 Jun, 2026.

COHEN, William. Determining whether an application has poor cache performance.

Red Hat Developer. 10 Mar. 2014. Disponível em:

<https://developers.redhat.com/blog/2014/03/10/determining-whether-an-application-has-poor-cache-performance-2#>. Acesso em: 05 Jun, 2026.

McVOY, Larry; STAELIN, Carl. Imbench: Portable Tools for Performance Analysis.

USENIX Annual Technical Conference (USENIX ATC 96), 1996, San Diego, CA.

USENIX Association. Jan. 1996. Disponível em:

https://www.usenix.org/legacy/publications/library/proceedings/sd96/full_papers/mcvoyn.pdf. Acesso em: 15 Jun, 2026.